

Software Architecture

(2025)

Outline

— [Software Architecture

— [Quality Attributes & Tactics

— [Architecture Patterns

— [Identifying ASRs

— [Designing Software
Architecture

— [Documenting Software
Architecture

— [Microservice Architecture

— [AI-native Application
Architecture

— [Domain-Driven Design

— [Enterprise & Business
Architecture

Software Architecture in General

- [What is software architecture?

- Structure, Elements, Relationships, Design

- [What does a software architect do? Liaison, Software engineering, Tech. knowledge, Risk mgmt.

- [Where do architectures come from?

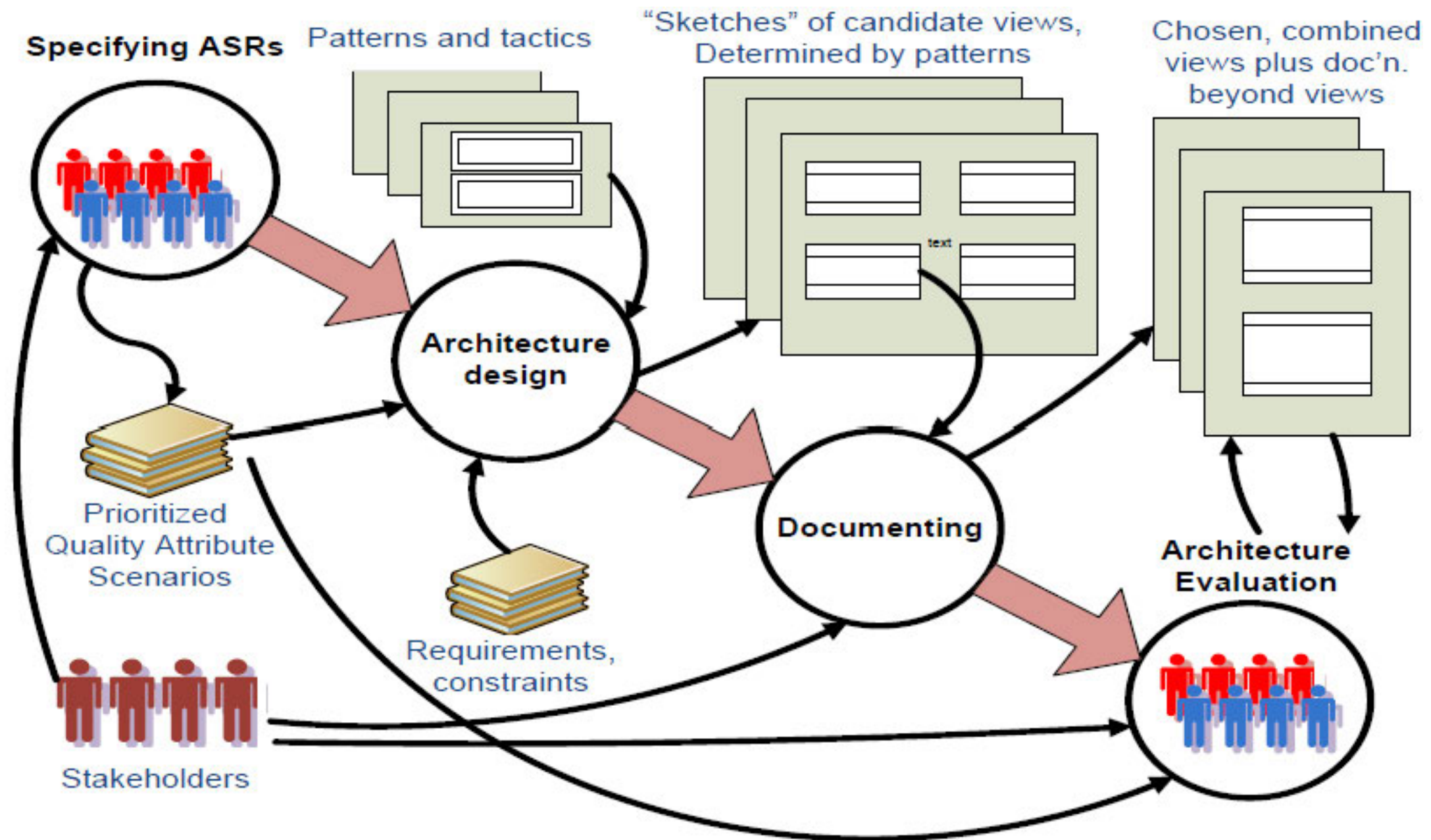
- NFRs, ASRs, Quality Requirements; Stakeholders, Organisations, Technical Environments...

- [Architecture (4+1) Views

- Logical view, Process view, Physical view, Development view + Use case scenarios...

- [Architectural activities and process

Architecture Process



Quality Attributes & Tactics

- [Software Requirements

- Functional requirements, Quality requirements ($\sim$ NFRs), Constraints

- [Quality Attributes & Tactics

- Internal vs. External Attributes

- Modeling quality attribute scenarios: Source, Stimulus, Artefact, Environment, Response, Measure

- Availability, Interoperability, Modifiability, Performance, Security, Testability, Usability, X-ability...

- Tactics for quality attributes

- Categories of architecture design decisions

Architecture Patterns

Architecture Patterns

- Context, Problem, Solution: elements + relations + constraints

Module Patterns

- Layered pattern (micro-kernel pattern)

Component-Connector Patterns

- Broker pattern, Model-view-controller pattern, Pipe-and-filter pattern, Client-server pattern, Peer-to-peer pattern, Service-oriented pattern, Publish-subscribe pattern, Share-data pattern

Allocation Patterns

- Map-reduce pattern, Multi-tier pattern

Patterns vs. Tactics

Identifying ASRs

— [Architecturally Significant Requirements

— Functional requirements, Quality requirements, Constraints

— [How to gather and identify ASRs?

— Requirement Docs, QA Workshop (interviews), Business goals

— Utility tree (Ranked scenarios: Importance vs. Difficulty)

Designing Architecture

- [General Design Strategy

- Abstraction, Decomposition, Divide & conquer, Generation and test, Iteration, Reuse

- [Categories of design decisions

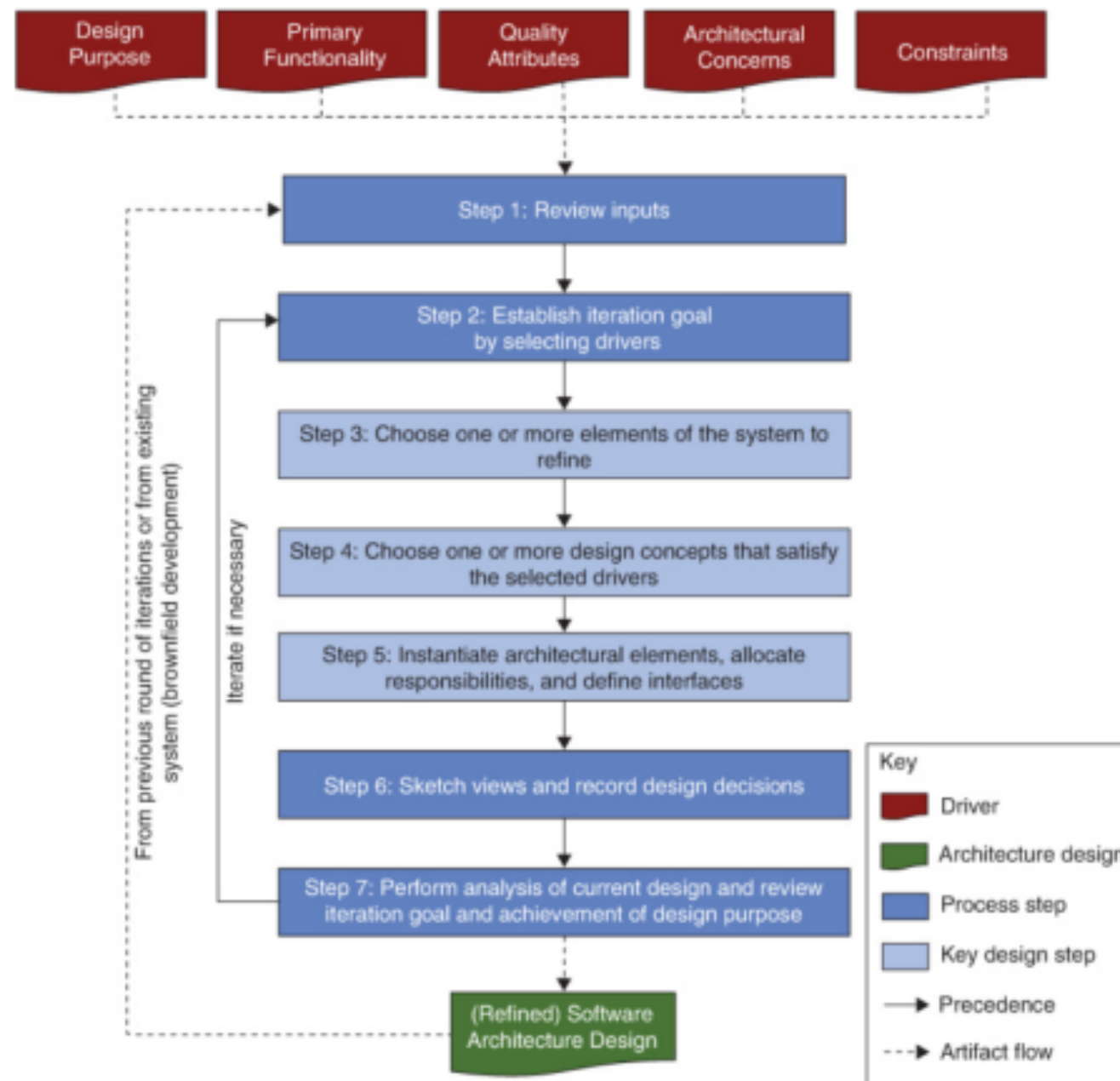
- Responsibilities, Coordination, Data, Resources, Elements mapping, Binding time, Technology

- [Architectural drivers

- Functional requirements, Quality attribute requirements, Constraints, The type of system that is being designed, Design objectives, Concerns

Designing Architecture

Attribute-Driven Design (ADD 3.0)



Documenting Architecture

Views and Beyond

Views:

- Styles (viewpoints), patterns and views

- Structural views: module views, component-and-connector views, allocation views

- Quality views

- Documenting views: 1. build stakeholder/view table, 2. combine views, 3. prioritise & stage

- Info. beyond views: documentation info & architecture info (mapping between views)

- Documentation package: views + beyond

Microservices Architecture

— [核心特征

- 微服务的六大特性、架构对比

— [微服务设计模式

- 挑战问题、模式和模式语言
- 拆分设计模式
- 通信模式
- 部署模式
- 运维模式.....

AI-native Application Architecture

核心特征

- 以模型驱动和数据为中心、Agent 协作、数据闭环、云原生集成

关键组件与工具链实践

- AI 网关（Higress）、配置与发现（Nacos）、可观测性（OTel）

典型场景与客户实践

- 内部工具（阿里云通义 App）、企业服务（零一万物）、工业场景...

未来规划

- 智能体平台、MCP 协议生态、多模态能力

Enterprise Architecture

— [核心定义

- 涵盖范围、结构化、分解战略目标、管理系统复杂度、标准化

— [方法论

- TOGAF、业务组件建模、数据建模

— [实践与工具

- 案例、建模/开发/治理工具

— [AI 融合

- 设计辅助、智能治理、分工升级

Domain-Driven Design

核心框架

- 战略设计：子领域，限界上下文，统一语言
- 战术设计：聚合，领域事件，资源库/工厂

关键实践与案例

- 设计原则、事件风暴建模、领域模型设计

AI 与 DDD 协同

- AI for DDD：领域模型、代码骨架与测试用例
- DDD for AI：为 AI 明确逻辑输入、简化AI 模型训练范围

Final Exam

——[总评成绩 = 平时成绩 * 40% + 期末考试 * 60%

——[内容占比：架构设计50% + 设计模式50%

——[题型：简答题、设计分析题

——[英文题目、中文或英文答题

——[个别题目可能需画图

——[基础内容60%

——[进阶内容40%